Fundamentals of Agentic AI

Aarthi Ravikrishnan

A*STAR Skin Research Laboratories (A*SRL)

14th October 2026

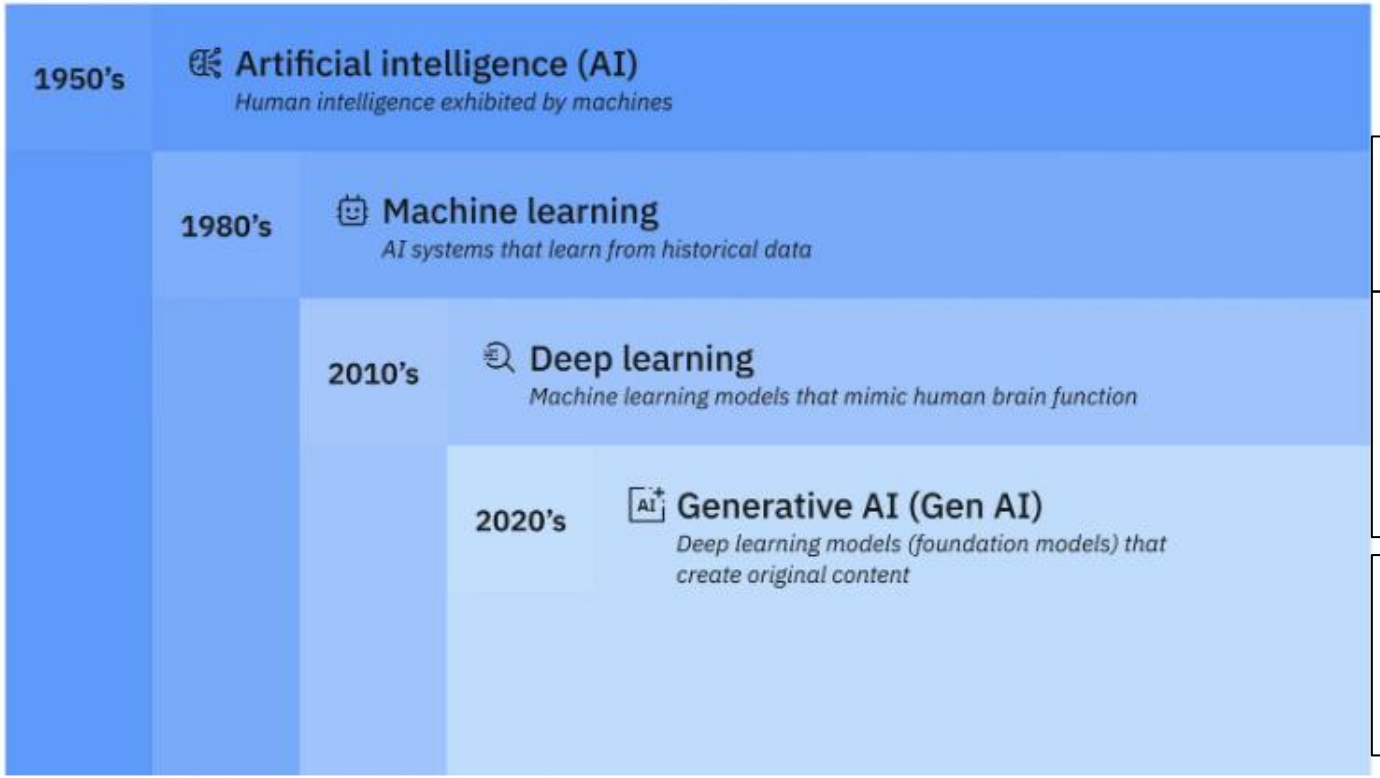
Champions of AI – Beginner workshop



Champions of AI

What exactly is AI?

- **AI – Technology that enables computers and machines to simulate human learning, comprehension, problem solving, decision making, creativity and autonomy.**



How artificial intelligence, machine learning, deep learning and generative AI are related.

Creating models by training an algorithm to make predictions or decisions based on the data. Eg: Linear/Logistic regression, Random Forest, SVM, KNN, clustering, Neural network

Multilayered neural networks, called deep neural networks, more closely simulate the complex decision-making power of human brain. These networks have: (i) Input layer (ii) 100s of hidden layers (iii) Output layer. Multiple layers → unsupervised learning → well-suited for ML at tremendous scale.

Generative AI -- **Deep learning models** that can create complex original content such as long-form text, high-quality images, realistic video or audio and more in response to a user's prompt or request.

Image from What Is Artificial Intelligence (AI)? | IBM

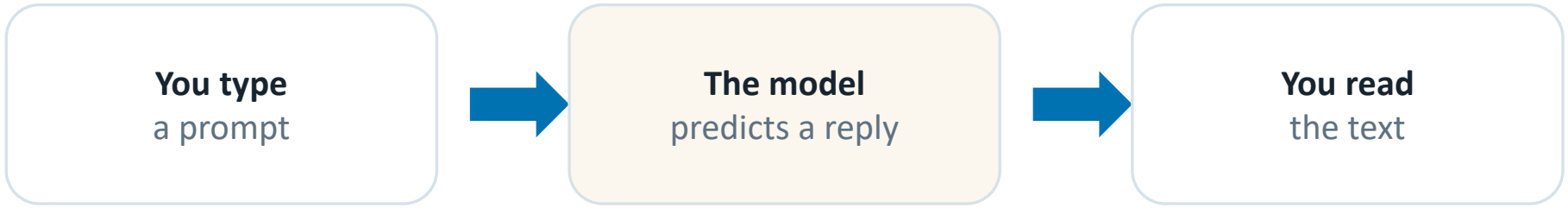
Is GenAI a new concept?

- Actually, no…
  - Have been used for years in statistics to analyse numerical data
- Generative model learns how data is produced, so it can both score data and create new data that looks like the real thing by modelling joint probability.
- Outputs → Stochastic
- Over last decade, 3 sophisticated deep learning model types emerged:
  1. Variational autoencoders – learn a compressed representation of data, then generate new examples by sampling from it
  2. Diffusion models – Models to generate original images in response to the prompts
  3. Transformers – Trained on sequenced data to generate extended sequences of content.
- Transformers are the core of most of today's genAI tool – ChatGPT, GPT-4, Copilot
- Transformers are the underlying neural network architecture that makes modern Large Language Models (LLM) possible.

So how different are LLMs and ML?



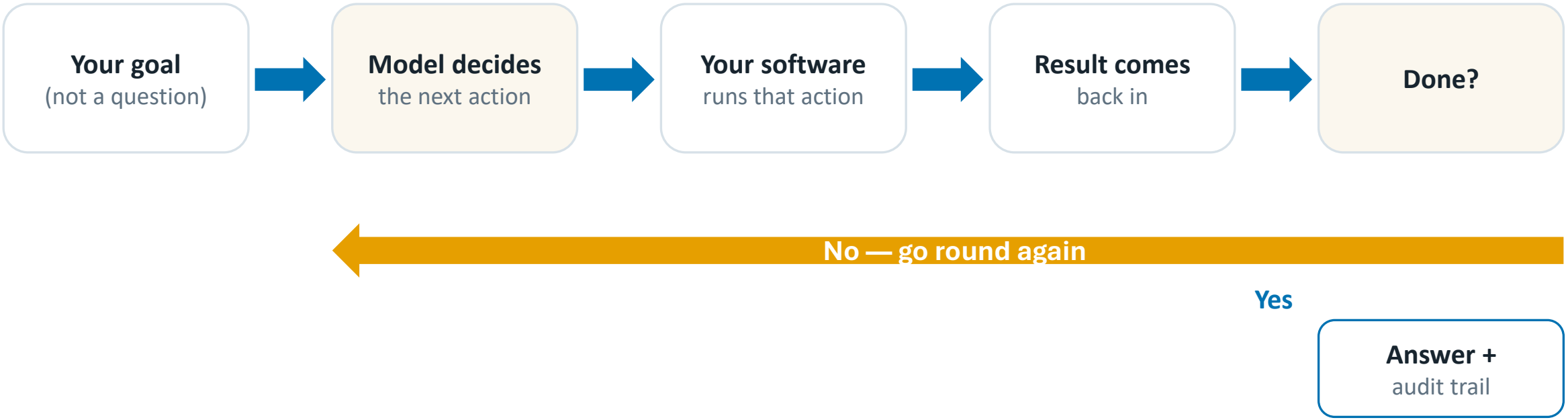
First: what a traditional LLM call actually is



One round trip. Then it is over.

- It answers from two things only: what you put in the prompt, and what it absorbed in training.
- It cannot look anything up, run anything, or check its own work.
- It stops when it has produced text — not when your task is finished.
- If more steps are needed, you do them. You read, you decide, you paste, you ask again.

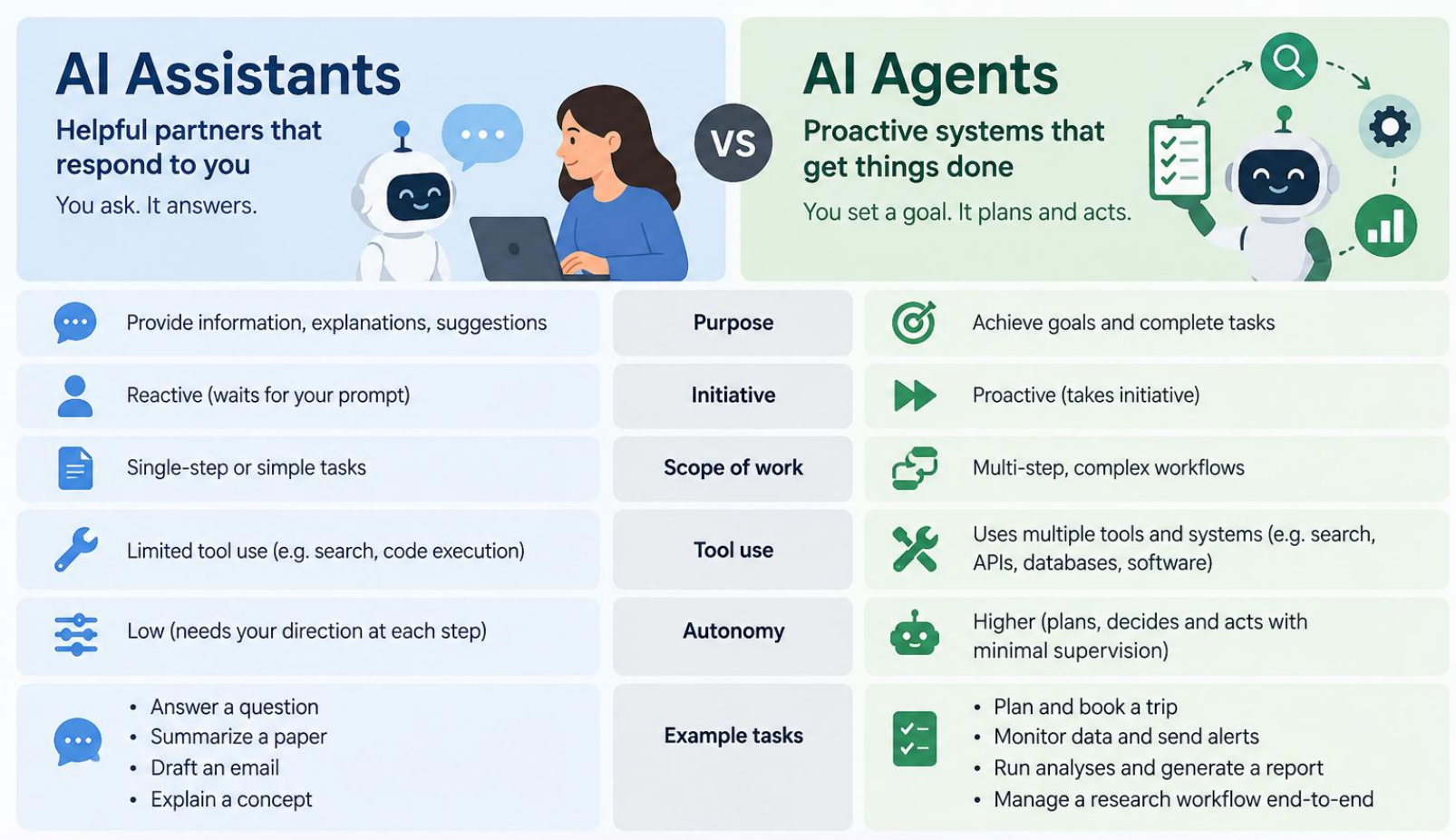
Agentic AI: the same model, put in a loop



Same model. What's new is the loop, the tools, and who decides the next step.

The model chose the path. Nobody scripted "search, then read, then tabulate." That choice is what the word "agentic" refers to.

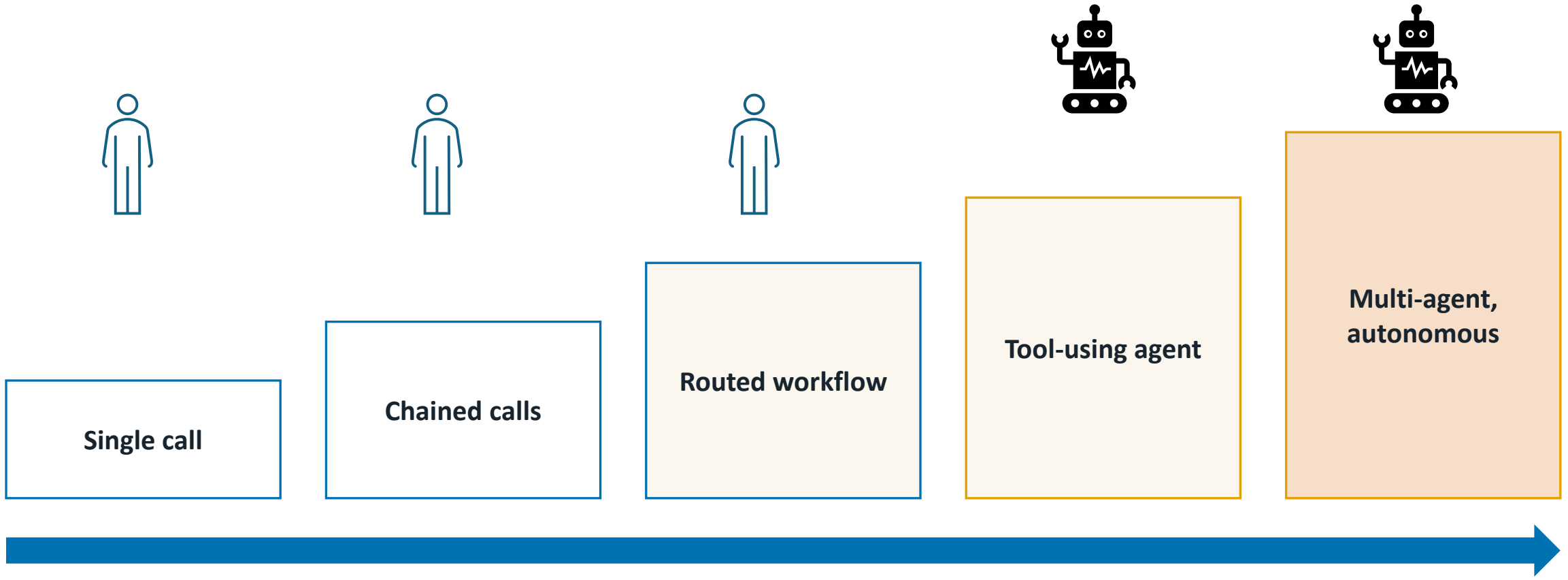
So now, what is an AI assistant vs AI agent?



ChatGPT, Gemini, Claude Chat

Claude with code execution

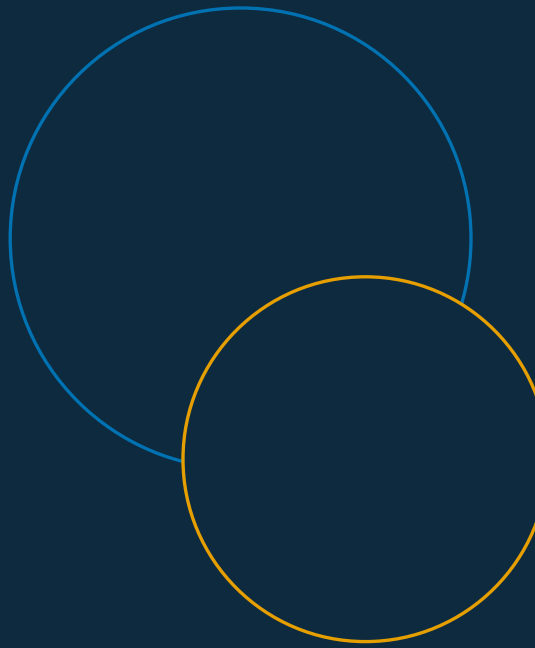
“Agentic” is a dial, not a switch



More autonomy → more it can handle, less you can predict

How one is built

Five parts, and the boundary that matters



HOW ONE IS BUILT

Anatomy of an agent

Model
The reasoning step. Chooses what to do next.

Instructions
Its role, its rules, what it must never do.

Tools
The only way it can touch anything real.

Memory
What it can still see as the task drags on.

Environment
Where it works: a sandbox, a folder, a database.

The loop + limits
Keeps going until done — or until your step/cost cap hits.

Swap the model and you get a better agent. Get the tools or the instructions wrong and no model saves you.

In practice teams spend most of their time on tools and context — not on the prompt, and not on picking the model.

What are the different kinds of models?

Term	Full Form	What It Does
LLM	Large Language Model	Processes and generates text. Examples: ChatGPT, Claude, Gemini.
SLM	Small Language Model	A smaller, more efficient version of an LLM that can run locally or on edge devices. Examples: Phi-3, Gemma 3, Llama 3.2 1B.
VLM	Vision-Language Model	Understands both images and text. Examples: GPT-4o, Claude Vision, Gemini.
MLM	Multimodal Language Model	Handles multiple modalities such as text, images, audio, video, and documents. Examples: GPT-4o, Gemini 2.5.
LCM	Large Concept Model	An emerging idea where models reason over concepts rather than predicting one token at a time. Meta has explored this direction in research.
LAM	Large Action Model	Designed to take actions rather than just generate text—e.g., use apps, click buttons, complete workflows. Rabbit's R1 was built around this concept.
MoE	Mixture of Experts	A model architecture where only a subset of expert networks is activated for each task, making very large models more efficient. Examples: Mixtral, GPT-4 family (believed to use MoE-like techniques), DeepSeek-V3.
SAM	Segment Anything Model	A computer vision model from Meta that can identify and segment objects in images with minimal prompting.

General progression of models

LLM → Understands language

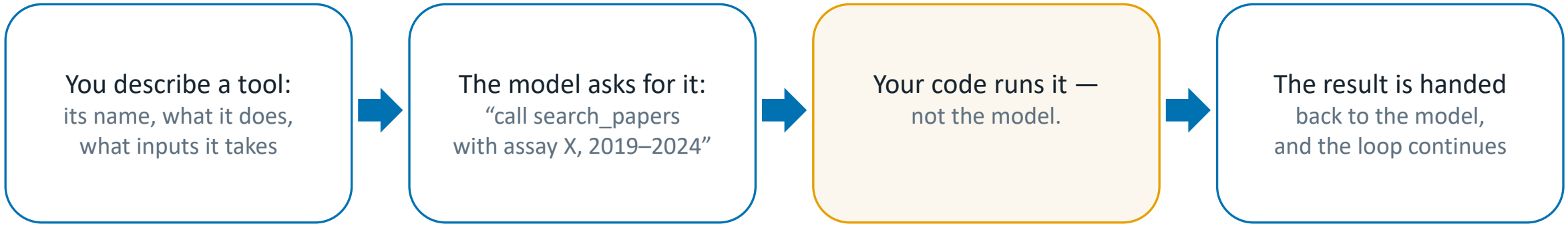
VLM → Understands language + images

MLM → Understands multiple data types

LAM → Can take actions

Agent → Uses LLM/VLM/LAM + tools + memory + planning

How a model “does” something: tools



Step 3 is the whole safety story: every real-world action passes through code you control.

- A “tool” is nothing exotic — a database query, a Python function, a web search, a file read, an email send.
- The model never gets credentials or a shell. It gets a menu, and it can only ask.
- MCP (Model Context Protocol) is the emerging standard for publishing that menu, so a tool you write once works with different agents — like a USB port for tools.

Memory: the constraint people underestimate

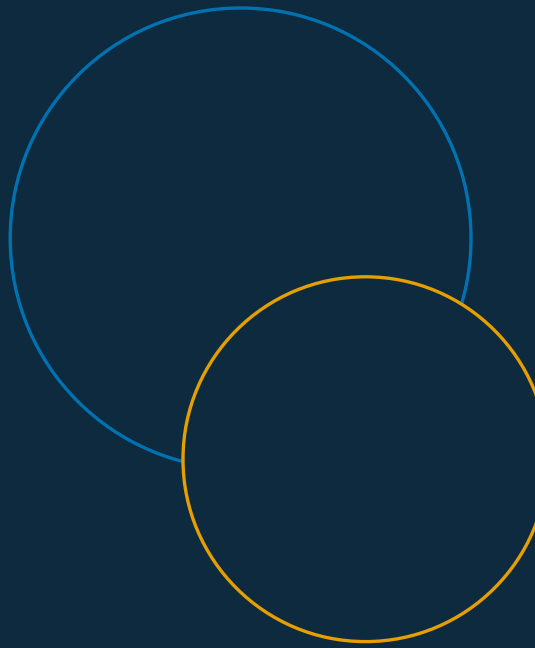
- **The context window is the agent's working memory**
  - Everything it currently knows — goal, history, tool results — sits in one buffer
- **Big windows don't mean reliable recall**
  - Accuracy degrades as it fills up. A million tokens is capacity, not comprehension.
- **So real agents manage it deliberately**
  - Summarise and compact old history; write findings to files and re-read on demand
  - Retrieve on demand instead of pre-loading everything “just in case”
- **This is why long-running agents keep notes**
  - External files are the durable memory; the context window is just the desk

Skill Files: Giving Agents Reusable Expertise

- Reusable workflows are stored as skills
- Skills are loaded only when needed
- Similar to SOPs in a laboratory
- Makes agents modular, maintainable, and scalable

Reality check

How these systems fail, and what to do about it



REALITY CHECK

How agents actually fail

- Goal drift — quietly solves a nearby, easier problem
- Error cascade — one bad intermediate result poisons every step after it
- Looping — retries the same failing action, burning budget
- Invented grounding — cites a paper, file, or function that doesn't exist
- Spec gaming — satisfies the letter of the request; deletes the failing test
- Agreeing with you — accepts a wrong premise instead of correcting it
- Silent partial success — reports “done” having finished 60%
- Notice the pattern: all of these look like success from the outside.

REALITY CHECK

Evaluation: the step everyone skips

- **Write the test set before you build the agent**
  - 20–50 realistic tasks where you already know what a good answer looks like
- **Judge two different things**
  - Outcome: was the final answer right? Path: did it get there sanely?
  - An agent that guessed right via three wrong turns will not survive a harder case
- **Automate the checkable parts**
  - Does the code run? Does the number match? Is the citation real?
- **Use a model as a grader for the rest — with a written rubric**
- **Log every run. You cannot debug what you cannot replay.**
- **Tuning prompts feels like progress. A score moving is progress.**

REALITY CHECK

The security problem with no clean fix

Prompt injection: text the agent reads can act as instructions it obeys.

Hidden in a web page, a PDF, an email, a code comment, a dataset description. The agent cannot reliably tell your instructions from content it encountered.

Private data
Your files, your database, your email

+

Untrusted content
The open web, inbound documents, third-party code

+

A way out
Email, HTTP calls, posting, writing to shared drives

All three at once = an agent that can be talked into exfiltrating your data. Remove any one and the attack collapses.

- Least privilege: read-only unless it truly needs to write. No standing credentials.
- A human approval gate on anything irreversible or outbound — sending, deleting, paying, publishing.
- Sandbox the environment; allowlist what it can reach; log every action with who authorised it.

REALITY CHECK

Before it touches real work

- **Know the data classification before anything leaves your control boundary**
  - A cloud-hosted model means your data goes to a third party — check what tier that's allowed for
- **Every agent needs a named owner and a kill switch**
- **Keep an audit trail: what it did, on whose behalf, with whose approval**
- **Set hard cost and step ceilings on day one, not after the first invoice**
- **Tell people when they're dealing with an agent, and give them a route to a human**

REALITY CHECK

When not to use an agent

- **The rules are known and stable → write ordinary software**
  - If you can specify the steps, a script is cheaper, faster and auditable
- **You can't check the output → you have no way to know it worked**
- **Mistakes are expensive and hard to spot → keep a human deciding**
- **It's one well-specified transformation → a single LLM call is plenty**
- **Tight latency or fixed cost budget → loops are variable by nature**
- **Most disappointing AI projects picked the wrong task, not the wrong tool.**

REALITY CHECK

Where this genuinely helps in research work

- **Good fits**
  - Literature triage and evidence tables, with sources attached
  - Data wrangling: messy files → tidy, with the code shown
  - Assembling analysis pipelines from existing tools
  - Exploratory analysis you were going to eyeball anyway
  - Drafting reports against documents you supply
- **What they share**
  - The output is checkable — run the code, open the citation
  - A wrong answer is cheap, and caught early
  - It's the tedious 80%, not the judgement call
  - A retry costs seconds, not a reputation
  - You'd have done it by hand otherwise

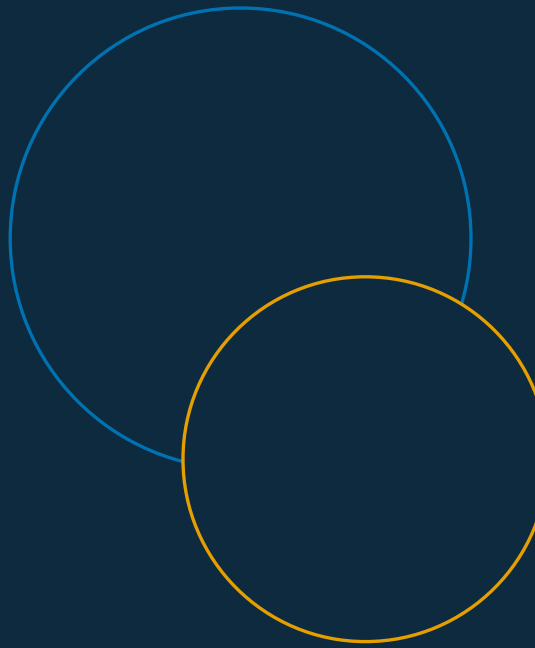
REALITY CHECK

Five things to remember

- It's the same model. The loop and the tools are what's new.
- In a single call, you are the loop. In an agent, the software is — and you supervise.
- Agency is a dial. Turn it up only when the simpler version has failed.
- Tools and context are the real engineering. The prompt is the easy part.
- Evaluation and permissions decide whether it survives contact with reality.

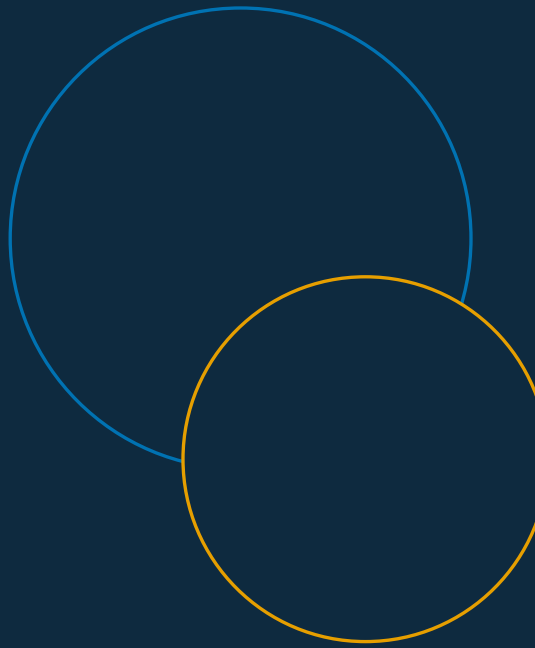
Thank you!

Any questions?



Hands-on

Let’s get our hands dirty! ☺



Some house-keeping notes

- We will be doing all our exercises on Claude Enterprise Desktop version
- We will use Sonnet 5.5 medium for our exercises
- Four hands-on exercises with different examples
- Underlying principles are the same → Can be extrapolated to other agentic AI tools such as MS Copilot
- Time permits → Demo on MS Copilot